Your Own Interfaces

A custom message, a service, parameters, and one deliberate deadlock

Prof. Anis Koubaa Asem Ibrahim Alalwan

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 3, Lecture B

How this session works

You type. We stop at every checkpoint until the room is together.

1. An interfaces package
2. A service, both halves
3. The deadlock, on purpose
4. Parameters
5. A first action client

Your Week 2 workspace, one new interfaces package, one new nodes package.

An interfaces package

EE 414 — Introduction to Robotics

Two packages, one for definitions

```
$ cd ~/ee414_ws/src
$ ros2 pkg create --build-type ament_cmake  ee414_w03_interfaces
$ ros2 pkg create --build-type ament_python \
      --dependencies rclpy std_msgs ee414_w03_nodes
```

Note the different build types. `ament_cmake` for the definitions, because interface generation is a CMake job; `ament_python` for your nodes, as last week.

Under the hood

Trying to put a `.msg` in your Python package is the first thing most people try. It builds, generates nothing, and the import fails at run time with a message that does not mention the cause.

Write the definitions

ee414_w03_interfaces/msg/WheelState.msg

```
std_msgs/Header  header
float64          left_rad_s
float64          right_rad_s
bool             estopped
```

ee414_w03_interfaces/srv/ResetOdom.srv

```
bool zero_heading
---
bool success
string message
```

Folders msg/ and srv/, exactly. Files CamelCase, fields snake_case — the generator enforces both.

Three lines in CMakeLists.txt

```
find_package(rosidl_default_generators REQUIRED)
find_package(std_msgs REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/WheelState.msg"
  "srv/ResetOdom.srv"
  DEPENDENCIES std_msgs
)
```

And two dependencies in `package.xml`. The exact lines are in the lab sheet — **copy them**. Nobody memorises this, including me.

`DEPENDENCIES std_msgs` is required because `WheelState` embeds a `Header`. Omit it and the build fails with a message about an unknown type.

Build, and check that it exists

```
$ cd ~/ee414_ws
$ colcon build --packages-select ee414_w03_interfaces
$ source install/setup.bash
$ ros2 interface show ee414_w03_interfaces/msg/WheelState
std_msgs/Header header
float64 left_rad_s
float64 right_rad_s
bool estopped
```

If `interface show` prints your fields, the generator ran and the Python class exists. If it says the interface is unknown, the build did not generate it — fix that before writing any node.

Checkpoint 1. Everyone can `interface show` their own message.

Use it from a node

```
from ee414_w03_interfaces.msg import WheelState
msg = WheelState()
msg.header.stamp = self.get_clock().now().to_msg()
msg.header.frame_id = 'base_link'
msg.left_rad_s  = 2.4
msg.right_rad_s = 2.4
msg.estopped = False
self.pub.publish(msg)
```

Import path is `package.msg`, then the type name. Fields are plain attributes.

Stamping is a habit, not a formality

`self.get_clock().now()` — never `time.time()`. In simulation the clock is not wall time, and a node using it disagrees with the whole robot. Week 8 shows this.

Echo your own type

```
$ ros2 topic echo /wheel_state
header:
  stamp:
    sec: 1755855012
  frame_id: base_link
left_rad_s: 2.4
right_rad_s: 2.4
estopped: false
```

Your own type, inspectable from the command line with no extra work — because you declared it properly rather than packing numbers into a string.

Checkpoint 2. Everyone sees their own fields in `echo`.

A service, both halves

EE 414 — Introduction to Robotics

The server

```python
from ee414_w03_interfaces.srv import ResetOdom

class OdomNode(Node):
    def __init__(self):
        super().__init__('odom_node')
        self.x = 12.5
        self.create_service(ResetOdom, 'reset_odom', self.on_reset)

    def on_reset(self, request, response):
        self.x = 0.0
        response.success = True
        response.message = 'odometry zeroed'
        return response
```

A service callback receives `request`, fills `response`, and **returns it**. Forgetting the return is the most common mistake — the caller then waits forever.

Test it before writing a client

```
$ ros2 service list
/reset_odom
$ ros2 service call /reset_odom \
      ee414_w03_interfaces/srv/ResetOdom "{zero_heading: true}"
response:
  success: True
  message: 'odometry zeroed'
```

One half at a time. The server is now proven. Anything that goes wrong from here is in the client — which is a much smaller place to look.

Checkpoint 3. Everyone has called their own service from the terminal.

The deadlock, on purpose

EE 414 — Introduction to Robotics

Write the version that hangs

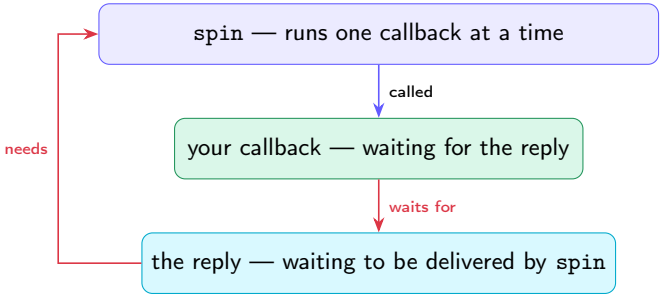
Add this to a *subscriber callback* — and be ready to press `Ctrl-C`:

```
def on_msg(self, msg):
    future = self.cli.call_async(ResetOdom.Request())
    rclpy.spin_until_future_complete(self, future)    <- HANGS
    self.get_logger().info('never printed')
```

It looks correct. It reads correctly. The node freezes, prints nothing more, and does not crash — which is the worst combination, because there is no error to search for.

Checkpoint 4. Everyone's node is now hung. Look at it for a moment.

Why it hangs, in one picture



A circle. Your callback is waiting for something only `spin` can deliver, and `spin` is inside your callback.

The fix: hand the waiting back to spin

```python
def on_msg(self, msg):
    future = self.cli.call_async(ResetOdom.Request())
    future.add_done_callback(self.on_reply)
    return

def on_reply(self, future):
    self.get_logger().info(f'reply: {future.result().message}')
```

The callback **returns immediately**. `spin` regains control, delivers the reply, and calls `on_reply`. Two short callbacks instead of one that waits.

In the field

The rule: a callback never waits. It does its work and returns; anything that takes time becomes a second callback. Internalise this and a whole class of ROS 2 hang stops happening to you.

Parameters

EE 414 — Introduction to Robotics

Declare, then read

```python
class Driver(Node):
    def __init__(self):
        super().__init__('driver')
        self.declare_parameter('max_speed', 0.22)
        self.declare_parameter('frame_id', 'base_link')
        self.create_timer(0.1, self.tick)

    def tick(self):
        v = self.get_parameter('max_speed').value
        self.get_logger().info(f'v = {v}')
```

Read the parameter **inside** `tick`, not once in `__init__`. That is what makes a live change take effect.

Change it while it runs

```
$ ros2 param list /driver
$ ros2 param get /driver max_speed
Double value is: 0.22
$ ros2 param set /driver max_speed 0.35
Set parameter successful
```

Watch the node's output change without restarting it. **That is tuning**, and it is how you will find your controller gains in Week 5 and your costmap sizes in Week 11.

Checkpoint 5. Everyone has changed a running node's behaviour from the terminal.

Where the values belong for good

```
Node(package='ee414_w03_nodes', executable='driver',
     parameters=[{'max_speed': 0.15, 'frame_id': 'base_link'}])
```

A value set from the terminal is gone when the node restarts. A value in a launch file is **reproducible** — and reproducibility is what a demo day needs.

The workflow

Tune live with `param set` until it behaves. Then write the value you found into the launch file. Students who skip the second step spend demo day wondering why the robot is worse than it was yesterday.

A first action client

EE 414 — Introduction to Robotics

Watch one work, from the terminal

```
$ ros2 run action_tutorials_py fibonacci_action_server

$ ros2 action list
/fibonacci
$ ros2 action send_goal /fibonacci \
      action_tutorials_interfaces/action/Fibonacci "{order: 10}" --feedback
Goal accepted with ID: 8a1f...
Feedback: partial_sequence: [0, 1, 1, 2, 3]
Result: sequence: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Run it **without** `--feedback` as well, and compare. The difference between the two runs is the entire reason actions exist.

The client, in the shape you already know

```
self.ac = ActionClient(self, Fibonacci, 'fibonacci')
self.ac.wait_for_server()

goal = Fibonacci.Goal()
goal.order = 10
future = self.ac.send_goal_async(goal, feedback_callback=self.on_fb)
future.add_done_callback(self.on_accepted)

def on_fb(self, fb):
    self.get_logger().info(f'progress: {fb.feedback.partial_sequence}')
```

`send_goal_async`, a future, a done callback — the same shape as the service client. One extra piece: a feedback callback, fired repeatedly while the server works.

Checkpoint 6. Everyone has seen feedback arrive from a running goal.

What you can now do that you could not last week

You built	**Which means**
Your own message type	Data travels as named typed fields, inspectable by tools you did not write
A service, both halves	A command you can confirm was received and succeeded
An async client	Callbacks that never block — the habit that prevents the hang
Parameters	Behaviour you can tune without editing or rebuilding
An action client	Long jobs you can watch and cancel

Together with Week 2, that is the whole of ROS 2's communication surface. **Everything from here is robotics.**

Before you leave

Leaving this room: an interfaces package that builds, a custom message published and echoed, a service called from both the terminal and a client, a deadlock you caused and fixed, a parameter changed live, and feedback from a running action.

Assignment 1 is due at the end of this week. Commit and push it.

Next week the robot moves

Week 4 spends its theory hour on differential-drive kinematics, and its second hour driving a TurtleBot3 in Gazebo with `/cmd_vel`. **Make sure Gazebo launches on your machine before then** — the setup guide has the command. Finding out in the session that it does not is an expensive way to spend ninety minutes.